



§. 习题课(Week-06/07)

1、双编译器的基本使用

- ★ VS配色方案的选择（工具-选项-环境-视觉体验-颜色主题）
- ★ VS“解决方案资源管理器”的位置（鼠标拖动的方法）
- ★ VS“解决方案资源管理器”被关闭（视图-解决方案资源管理器）
- ★ VS一个解决方案包含多个项目（后续作业推荐使用此方案）
- ★ 小熊猫设定VS的解决方案目录（方便打开对应文件）



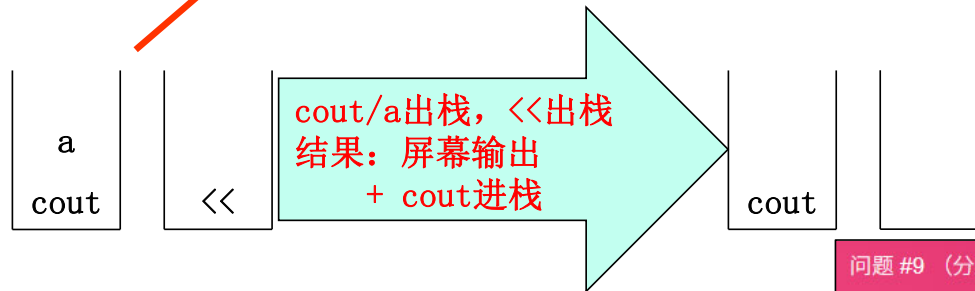
§. 习题课 (Week-06/07)

2、运算符优先级与结合性

★ 含有cin/cout的表达式分析

- cout、endl当运算数
- << 当运算符
- **cout << 表达式**的计算结果为cout

例: `cout << a << b << endl;` (<<是7级优先级, 左结合)



正在进行

W0302-cin-cout知识点回顾 (非作业)

基础知识题 (共0分) ^

8 9 10 11 12 13

问题 #9 (分数0)

```
#include <iostream>
using namespace std;

int main()
{
    int a=10, b=15, c=20;
    cout << a, b, c;
    return 0;
}
```

问题 #1: 这个程序的输出为什么只有一个10?

问题 #10 (分数0)

```
#include <iostream>
using namespace std;

int main()
{
    int a=10, b=15, c=20;
    cout << (a, b, c) << endl;
    return 0;
}
```

这个程序编译出什么错? 贴错误截图并从优先级结合性角度解释错误原因

问题 #1: 这个程序的输出为什么只有一个20?

问题 #11 (分数0)

```
#include <iostream>
using namespace std;

int main()
{
    int a=10, b=15, c=20;
    cout << a, b, c << endl;
    return 0;
}
```



§. 习题课 (Week-06/07)

2、运算符优先级与结合性

★ 含有cin/cout的表达式分析

★ 含有()的表达式分析

作业已结束

W0301-变量及表达式求值

问题 #23 (分数8)

仿照第2章课件的P.69-118，用栈方式给出下列表达式的求解过程

$a + (a - 3 / (b + c) + 5) \% 4$ (假设所有变量均为int型)

表达式一共有10个运算符，因此8个步骤分别是：

步骤①：__1__

步骤②：__2__

步骤③：__3__

步骤④：__4__

步骤⑤：__5__

步骤⑥：__6__

步骤⑦：__7__

步骤⑧：__8__

注：左右括号消除算一步，上面8个步骤中有两步是左右括号消除，其余6步是计算步骤



§. 习题课 (Week-06/07)

3、IEEE754相关

★ 补码的数学解析

● 模与补数:

模: 某种类型的数据能够表示的范围(例: 1字节数据, 模=256)

=> 数据超过模的范围则自动取模(溢出截断)

补数: 一个数的补数等于模-自身 (例: 数据 补数)

10	246
127	129

● 减法的两个方法:

◆ 正常减法: $A - B$

◆ 补数加法: $A + B$ 的补数

例: $100 - 10 = 90$

$100 + 246 \bmod 256 = 90$ (溢出截断)

=> 计算机内所有减法都可以表示为加法

● 负数的补码当做无符号数理解, 正好就是其补数

(两者绝对值相加为模)

-10 : 绝对值 -> 0000 1010

取反 -> 1111 0101

+1 -> 1111 0110 = $2^7 + 2^6 + 2^5 + 2^4 + 2^2 + 2^1$
= $128+64+32+16+4+2 = 246$

§. 习题课

3、IEEE754相关

★ 三个特殊的值: 0/inf/nan

0:

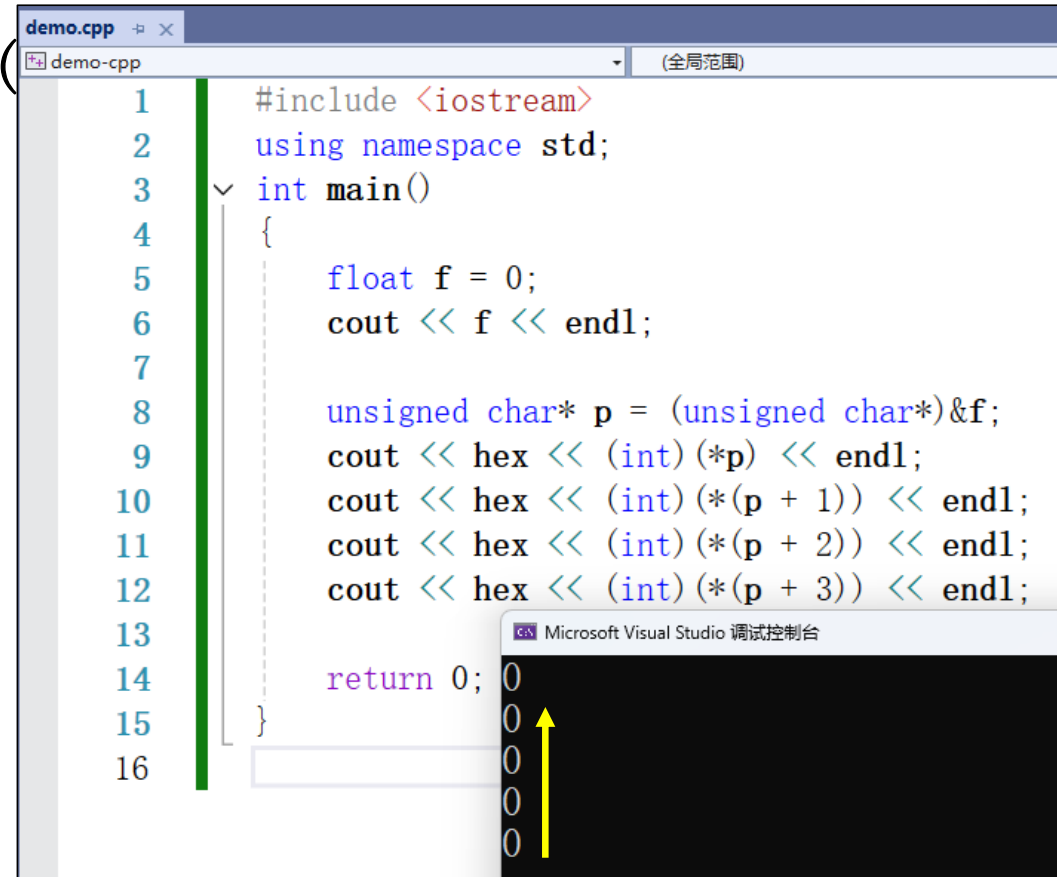
- 指数部分全0, 尾数部分全0, 则表示0
此时不能理解为 $0.0 * 2^{-127}$ (23位位数为0)
 $1.0 * 2^{-127}$ (补隐含的1)

```
#include <iostream>
using namespace std;
int main()
{
    float f = 0;
    cout << f << endl;

    unsigned char* p = (unsigned char*)&f;
    cout << hex << (int)(*p) << endl;
    cout << hex << (int)*(p+1) << endl;
    cout << hex << (int)*(p+2) << endl;
    cout << hex << (int)*(p+3) << endl;

    return 0;
}
```

0000 0000 0000 0000 0000 0000 0000 0000



```
demo.cpp demo-cpp (全局范围)
1 #include <iostream>
2 using namespace std;
3 int main()
4 {
5     float f = 0;
6     cout << f << endl;
7
8     unsigned char* p = (unsigned char*)&f;
9     cout << hex << (int)(*p) << endl;
10    cout << hex << (int)*(p + 1) << endl;
11    cout << hex << (int)*(p + 2) << endl;
12    cout << hex << (int)*(p + 3) << endl;
13
14    return 0;
15 }
16
```

Microsoft Visual Studio 调试控制台

0
0
0
0
0

§. 习题课 (Week-06/07)

3、IEEE754相关

★ 三个特殊的值: 0/inf/nan

inf: 无穷大(infinite)

- 指数部分全1, 尾数部分全0, 则表示无穷大, 可根据符号位分正负
- 浮点数本身范围有限, 虽然可表示无穷大, 但无法表示可增长到
- double型自行测试

0111 1111 1000 0000 0000 0000 0000 0000

1111 1111 1000 0000 0000 0000 0000 0000

```
#include <iostream>
using namespace std;
int main()
{
    float f = 1e40; //超float上限
    cout << f << endl;

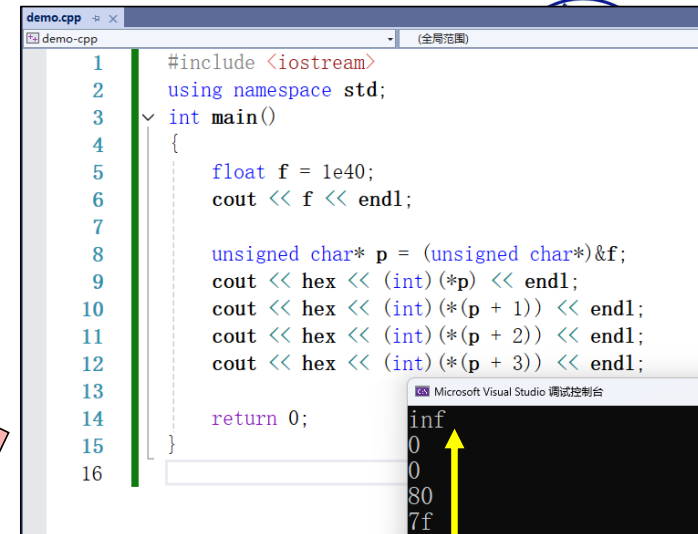
    unsigned char* p = (unsigned char*)&f;
    cout << hex << (int)(*p) << endl;
    cout << hex << (int)*(p+1) << endl;
    cout << hex << (int)*(p+2) << endl;
    cout << hex << (int)*(p+3) << endl;

    return 0;
}
```

```
#include <iostream>
using namespace std;
int main()
{
    float f = -1e40; //超float下限
    cout << f << endl;

    unsigned char* p = (unsigned char*)&f;
    cout << hex << (int)(*p) << endl;
    cout << hex << (int)*(p+1) << endl;
    cout << hex << (int)*(p+2) << endl;
    cout << hex << (int)*(p+3) << endl;

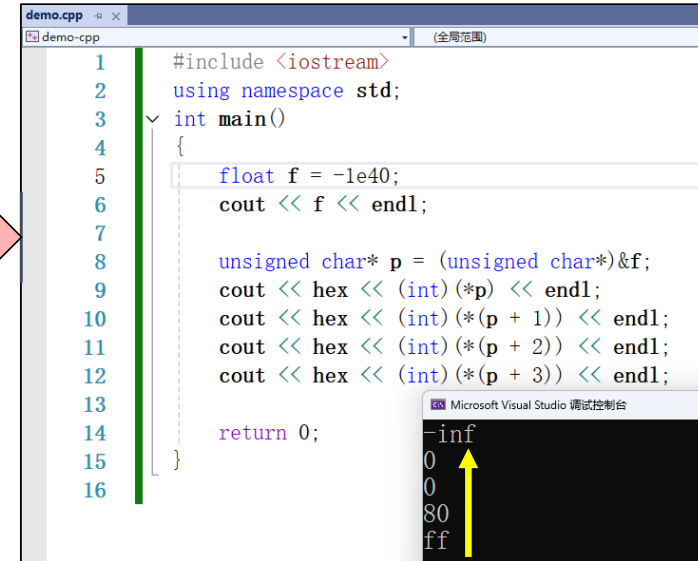
    return 0;
}
```



```
demo.cpp
1 #include <iostream>
2 using namespace std;
3 int main()
4 {
5     float f = 1e40;
6     cout << f << endl;
7
8     unsigned char* p = (unsigned char*)&f;
9     cout << hex << (int)(*p) << endl;
10    cout << hex << (int)*(p + 1) << endl;
11    cout << hex << (int)*(p + 2) << endl;
12    cout << hex << (int)*(p + 3) << endl;
13
14    return 0;
15 }
16
```

Microsoft Visual Studio 调试控制台

inf
0
0
80
7f



```
demo.cpp
1 #include <iostream>
2 using namespace std;
3 int main()
4 {
5     float f = -1e40;
6     cout << f << endl;
7
8     unsigned char* p = (unsigned char*)&f;
9     cout << hex << (int)(*p) << endl;
10    cout << hex << (int)*(p + 1) << endl;
11    cout << hex << (int)*(p + 2) << endl;
12    cout << hex << (int)*(p + 3) << endl;
13
14    return 0;
15 }
16
```

Microsoft Visual Studio 调试控制台

-inf
0
0
80
ff



§. 习题课 (Week-06/07)

3、IEEE754相关

★ 三个特殊的值: 0/inf/nan

nan: 非法(not a number)

ind: indeterminate

● 指数部分全1, 尾数部分不全0, 则表示非法, 符号位无意义

1111 1111 1100 0000 0000 0000 0000 0000

```
#include <iostream>
using namespace std;
int main()
{
    float f = sqrt(-2); //负数开根号非法
    cout << f << endl;

    unsigned char* p = (unsigned char*)&f;
    cout << hex << (int)(*p) << endl;
    cout << hex << (int)*(p+1) << endl;
    cout << hex << (int)*(p+2) << endl;
    cout << hex << (int)*(p+3) << endl;

    return 0;
}
```

```
demo.cpp
demo.cpp (全局范围)
1 #include <iostream>
2 using namespace std;
3 int main()
4 {
5     float f = sqrt(-2); //负数开根号非法
6     cout << f << endl;
7
8     unsigned char* p = (unsigned char*)&f;
9     cout << hex << (int)(*p) << endl;
10    cout << hex << (int)*(p + 1) << endl;
11    cout << hex << (int)*(p + 2) << endl;
12    cout << hex << (int)*(p + 3) << endl;
13
14    return 0;
15 }
16

Microsoft Visual Studio 调试控制台
-nan(ind)
0
0
c0
ff
```



§. 习题课 (Week-06/07)

3、IEEE754相关

★ 有效位数的理解

- 有效位数的数学定义：从左边第一个非零数字起的数字位数
- 对float型数据，23位尾数，加隐含的1，则尾数的最大值(二进制)
 $1111\ 1111\ 1111\ 1111\ 1111\ 1111 = 16777216\ (2^{24})$
可以表示0~9999999的所有十进制7位数，但8位数不能超过16777216
=> float型数的有效位数为7位
=> 浮点数表示时，一般表示为 $x.xxxxxx * 10^{xx}$ ，也称小数点后6位
=> 综上：各种资料一般都称float的有效数字为6~7位
- 对double型数据，52位位数，加隐含的1，则位数的最大值(二进制)
 $9,007,199,254,740,992\ (2^{53})$ 是一个16位的十进制数
=> double型数的有效位数为15位
=> 因为接近能将16位整数全部表示完，有时也称有效位数16位
=> 综上：各种资料一般都称float的有效数字为15~16位
- **本课程统一**：float的有效位数7位，double的有效位数15位

§. 习题课 (Week-06/07)



3、IEEE754相关

★ 浮点数的输出

- 超出有效位数的可以输出，但**不可信**

=> 按前页，float型数的有效位数为前7位100%可信，第8位只有16%可信

=> 按前页，double型数的有效位数为前15位100%可信，第16位只有90%可信



§. 习题课 (Week-06/07)

3、IEEE754相关

★ 浮点数误差控制-“四舍六入五成双”(奇进偶舍)

- 四舍五入的弊病：单向进位导致系统性偏大，容易产生累积性误差

- 四舍六入五成双的基本规则

- ◆ 尾数 ≤ 4 时：舍去(1.3249 $\Rightarrow$ 1.32)

假设所有例子均保留小数点后2位

- ◆ 尾数 ≥ 6 时：进位(1.3267 $\Rightarrow$ 1.33)

- ◆ 尾数=5 时：判断5后面的数字及5的前一位的奇偶性

- ▲ 5后有非零数：进位(1.33501 $\Rightarrow$ 1.34 1.3251 $\Rightarrow$ 1.33)

- ▲ 5后全0或无数字：5前一位数字为奇数-进位(1.3350 $\Rightarrow$ 1.34)

- 5前一位数字为偶数-舍去(1.3250 $\Rightarrow$ 1.32)

- 和进制无关



§. 习题课 (Week-06/07)

4、cin相关

★ cin的基本步骤

Step1: 用户输入数据(字符序列)到缓冲区中(采用队列方式)

Step2: 按**合理且最长原则**读缓冲区, 直到回车/空格/非法输入为止

Step3: 将读到的字符序列转为变量类型, 如果转换过程中超过变量的范围, 则置good()为0/fail()为1

§ 3. 结构化程序设计基础

3.4. C++的输入与输出

3.4.3. 输入流的基本操作

格式: cin >> 变量1 >> 变量2 >> ... >> 变量n;

★ 输入终止条件为回车、空格、非法输入

★ 系统会自动根据cin后变量的类型按**合理且最长原则**来读取数据

★ 变量读取后, 系统会判断输入数据是否超过变量的范围, 若超过则置**内部的错误标记**并返回一个**不可信**的值(不同编译器处理不同)

★ cin输入完成后, 通过cin.good()/cin.fail()可判断本次输入是否正确

第3章课件P. 23

输入	cin.good() 返回	cin.fail() 返回
正确范围 +回车/空格/非法输入	1	0
错误范围 +回车/空格/非法输入	0	1
非法输入	0	1



§. 习题课 (Week-06/07)

4、cin相关

★ cin的基本步骤

★ 如何理解合理且最长原则

	第一个字符	后续字符	非法字符
整型(以十进制为例)	+, -, 0~9	0~9	非0~9 (包括+, -)
浮点型(以小数形式为例)	+, -, 小数点, 0~9	0~9及第一个出现的小数点	非0~9 (包括+, -, 小数点)
字符	缓冲区中第一个可读字符	无	不含空格、回车等不可读字符
字符串(暂不考虑)			

- 按数学常识理解即可, 不需要额外知识
- 第一个非法则置good()为0/fail()为1并返回

例1:

```
int a;
```

```
cin >> a;    //键盘输入1234567890123456.12
```

处理: 读到. 为止, 超上限, good()为0, 值不可信

例2:

```
char a;
```

```
cin >> a;    //键盘输入1234567890123456.12
```

处理: 读到2为止, 只读到一个1, 值为ASCII值49



§. 习题课 (Week-06/07)

4、cin相关

- ★ cin的基本步骤
- ★ 如何理解合理且最长原则
- ★ 连续多个输入的理解

例1:

📺 正在进行

W0302-cin-cout知识点回顾 (非作业)

问题 #28 (分数0)

```
#include <iostream>
using namespace std;
int main()
{
    char c1, c2;
    int a;
    float b;
    cin >> c1 >> c2 >> a >> b;

    cout << c1 << ' ' << c2 << ' ' << a << ' ' << b << endl;
    return 0;
}
```

注：~表示空格

1、输入：1234~56.78✓

输出：

2、输入：1~2~34~56.78✓

输出：

3、分析在以上两种不同输入的情况下，
为什么输出相同(提示：空格的作用)



§. 习题课 (Week-06/07)

4、cin相关

- ★ cin的基本步骤
- ★ 如何理解合理且最长原则
- ★ 连续多个输入的理解

例2:

```
int a;
```

```
char b;
```

```
cin >> a >> b; //键盘输入12.3✓
```

处理: a读到.为止, 得值12, 缓冲区中剩余.3✓, b读到., 缓冲区中还剩余3✓ (程序运行完成后丢弃)

```
demo.cpp  (全局范围)  main()
1  #include <iostream>
2  using namespace std;
3
4  int main()
5  {
6      int a;
7      char b;
8      cin >> a >> b; //键盘输入12.3✓
9      cout << cin.good() << ', ' << a << ', ' << b << endl;
10     return 0;
11 }
12
```

Microsoft Visual Studio 调试控制台

```
12.3
1 12 .
```



§. 习题课 (Week-06/07)

4、cin相关

- ★ cin的基本步骤
- ★ 如何理解合理且最长原则
- ★ 连续多个输入的理解

例3:

```
int a;
```

```
float b;
```

```
cin >> a >> b; //键盘输入12.3✓
```

处理: a读到.为止, 得值12, 缓冲区中剩余.3✓, b读到.3, 得值0.3, 缓冲区中还剩余✓ (程序运行完成后丢弃)

```
demo.cpp demo-cpp (全局范围) main()
1 #include <iostream>
2 using namespace std;
3
4 int main()
5 {
6     int a;
7     float b;
8     cin >> a >> b; //键盘输入12.3✓
9     cout << cin.good() << ' ' << a << ' ' << b << endl;
10    return 0;
11 }
12
```

Microsoft Visual Studio 调试控制台

```
12.3
1 12 0.3
```



§. 习题课 (Week-06/07)

4、cin相关

- ★ cin的基本步骤
- ★ 如何理解合理且最长原则
- ★ 连续多个输入的理解

例4:

```
int a, b;
```

```
cin >> a >> b; //键盘输入12.3✓
```

处理: a读到.为止, 得值12, 缓冲区中剩余.3✓

b读到., 第一个即非法, 置good()为0, b被赋值但不可信(VS下b为0), 缓冲区中还剩余.3✓
(在good()恢复为1前不再读任何数据)

```
demo.cpp  (全局范围)  main()
1  #include <iostream>
2  using namespace std;
3
4  int main()
5  {
6      int a, b;
7      cin >> a >> b; //键盘输入12.3✓
8      cout << cin.good() << ' ' << a << ' ' << b << endl;
9      return 0;
10 }
11
```

Microsoft Visual Studio 调试控制台

```
12.3
0 12 0
```




§. 习题课 (Week-06/07)

4、cin相关

- ★ cin的基本步骤
- ★ 如何理解合理且最长原则
- ★ 连续多个输入的理解

例5:

```
int a=11, b=22, c=33;
```

```
cin >> a >> b >> c; //键盘输入12.3✓
```

处理: a读到.为止, 得值12, 缓冲区中剩余.3✓

b读到., 第一个即非法, 置good()为0, b被赋值但不可信(VS下b为0), 缓冲区中还剩余.3✓

(在good()恢复为1前不再读任何数据, 因此c维持原值33)

```
demo.cpp demo-cpp (全局范围) main()
1 #include <iostream>
2 using namespace std;
3
4 int main()
5 {
6     int a = 11, b = 22, c = 33;
7     cin >> a >> b >> c; //键盘输入12.3✓
8     cout << cin.good() << ' ' << a << ' ' << b << ' ' << c << endl;
9     return 0;
10 }
11
```

Microsoft Visual Studio 调试控制台

```
12.3
0 12 0 33
```



§. 习题课 (Week-06/07)

4、cin相关

- ★ cin的基本步骤
- ★ 如何理解合理且最长原则
- ★ 连续多个输入的理解

例5: 变化

```
int a, b, c;
```

```
cin >> a >> b >> c; //键盘输入12.3✓
```

处理: a读到.为止, 得值12, 缓冲区中剩余.3✓

b读到., 第一个即非法, 置good()为0, b被赋值但不可信(VS下b为0), 缓冲区中还剩余.3✓

(在good()恢复为1前不再读任何数据, 因此c维持原值, 若未初始化则为不确定的随机值)

```
demo.cpp 1 #include <iostream>
2 using namespace std;
3
4 int main()
5 {
6     int a, b, c;
7     cin >> a >> b >> c; //键盘输入12.3✓
8     cout << cin.good() << ' ' << a << ' ' << b << ' ' << c << endl;
9     return 0;
10 }
11
```

Microsoft Visual Studio 调试控制台

```
12.3
0 12 0 -858993460
```

§. 习题课 (Week-

4、cin相关

- ★ cin的基本步骤
- ★ 如何理解合理且最长原则
- ★ 连续多个输入的理解
- ★ 不可信值的理解

- 只要good()不为1，值就不可信

```
#include <iostream>
using namespace std;

int main()
{
    short a;
    cin >> a;
    cout << cin.good() << ' ' << a << endl;

    return 0;
}
```



The screenshot shows the Visual Studio IDE with the file demo.cpp open. The code is as follows:

```
1 #include <iostream>
2 using namespace std;
3
4 int main()
5 {
6     short a;
7     cin >> a;
8     cout << cin.good() << ' ' << a << endl;
9
10    return 0;
11 }
```

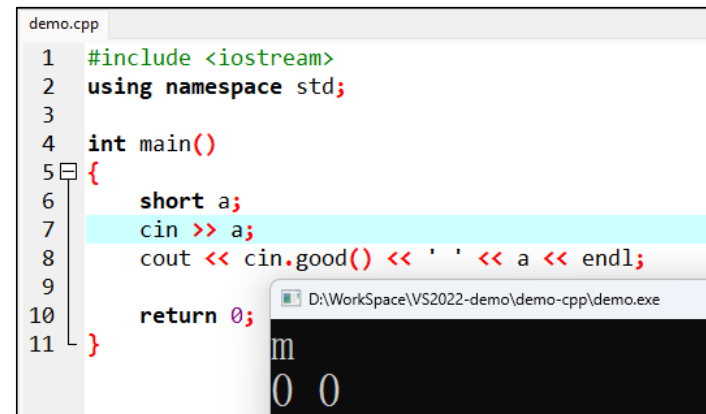
The output window at the bottom shows the result of running the program:

```
m
0 0
```

```
[root@RH9-DEV root]# cat test.cpp
#include <iostream>
using namespace std;
int main()
{
    short k;
    cin >> k;
    cout << "k=" << k << endl;
    cout << "cin.good()=" << cin.good() << endl;
    cout << "cin.fail()=" << cin.fail() << endl;
    return 0;
}
```

某Linux编译器

```
[root@RH9-DEV root]# ./test
m
k=16385
cin.good()=0
cin.fail()=1
```



The screenshot shows the Visual Studio IDE with the file demo.cpp open. The code is as follows:

```
1 #include <iostream>
2 using namespace std;
3
4 int main()
5 {
6     short a;
7     cin >> a;
8     cout << cin.good() << ' ' << a << endl;
9
10    return 0;
11 }
```

The output window at the bottom shows the result of running the program:

```
m
0 0
```



§. 习题课 (Week-06/07)

4、cin相关

- ★ cin的基本步骤
- ★ 如何理解合理且最长原则
- ★ 连续多个输入的理解
- ★ 不可信值的理解
 - 只要good()不为1, 值就不可信
 - 只要good()为1则可信 (无符号数读入负数问题)

```
#include <iostream>
using namespace std;
int main()
{
    unsigned short a;
    cin >> a;
    cout << cin.good() << ' ' << a << endl;
    return 0;
}
```

```
demo.cpp demo-cpp (全局范围)
1 #include <iostream>
2 using namespace std;
3 int main()
4 {
5     unsigned short a;
6     cin >> a;
7     cout << cin.good() << ' ' << a << endl;
8     return 0;
9 }
10
```

Microsoft Visual Studio 调试控制台

-12345
1 53191

1100 1111 1100 0111
有符号理解: -12345
无符号理解: 53191
cin读入后按有符号转为二进制,
赋值给无符号, 不超限即可 (-65535~-1)



§. 习题课 (Week-06/07)

5、其他

★ 除数/模数为0的问题

- 整数的除数/模数为0 : 编译error/运行出错
- 浮点数的除数为0 : 编译warning/运行结果为inf

作业已结束

W0301-变量及表达式求值

问题 #33 (分数5)

求复合赋值表达式的值 (要求给出计算过程、每步计算结果中变量的值、**不需要截图**)

已知: `int a = 8, n = 13;`

求 : `n %= a %= 4`

注: 本题需要解释, 为什么编译不报错, 但运行无输出、返回代码为负值、且运行时间比前几题长
(无法理解或说清楚原因的, 给出合理猜测也可, **直接回答不会也行, 到时习题课好好听**)

```
demo.cpp  X
demo.cpp  (全局范围)
1  #include <iostream>
2  using namespace std;
3  int main()
4  {
5      int a = 8, n = 13;
6      cout << (n %= a %= 4) << endl;
7      return 0;
8  }
9
10
11
```

Microsoft Visual Studio 调试控制台

E:\Workspace\VS2026-Demo\x64\Debug\demo-cpp.exe (进程 28528)已退出, 代码为 -1073741676 (0xc0000094)。
按任意键关闭此窗口. . .



§. 习题课 (Week-06/07)

5、其他

★ 除数/模数为0的问题

- 整数的除数/模数为0：观察除数/模数为变量/常量(包括符号常量)的不同表现
- 浮点数的除数为0

为什么编译器处理不同？
编译器思维的培养

```
demo.cpp (全局范围)
1 #include <iostream>
2 #include <iomanip>
3 using namespace std;
4
5 int main()
6 {
7     int a = 0;
8
9     cout << 10/a << endl;
10
11     return 0;
12 }
```

运行错

Microsoft Visual Studio 调试控制台

D:\Workspace\VS2022-demo\Debug\demo-cpp.exe (进程 16544) 已退出, 代码为 -1073741678 (0xc0000094)。

```
demo.cpp (全局范围)
1 #include <iostream>
2 #include <iomanip>
3 using namespace std;
4
5 int main()
6 {
7     const int a = 0;
8
9     cout << 10/a << endl;
10
11     return 0;
12 }
```

编译错

被零除或对零求模

```
demo.cpp (全局范围)
1 #include <iostream>
2 #include <iomanip>
3 using namespace std;
4
5 int main()
6 {
7     cout << 10%0 << endl;
8
9     return 0;
10 }
```

编译错

被零除或对零求模



§. 习题课 (Week-06/07)

5、其他

★ 除数/模数为0的问题

- 整数的除数/模数为0：观察除数/模数为变量/常量(包括符号常量)的不同表现
- 浮点数的除数为0：观察除数为变量/常量(包括符号常量)的不同表现

```
#include <iostream>
using namespace std;
int main()
{
    float f1 = 1.2F;
    cout << f1/0 << endl;

    return 0;
}
```

```
#include <iostream>
using namespace std;
int main()
{
    float f1 = 1.2F;
    float f2 = 0;
    cout << f1/f2 << endl;
    return 0;
}
```

The screenshot shows the Visual Studio IDE with a C++ file named `demo.cpp`. The code is as follows:

```
1 #include <iostream>
2 using namespace std;
3 int main()
4 {
5     float f1 = 1.2F;
6     cout << f1 / 0 << endl;
7
8     return 0;
9 }
```

The output window shows the following message:

```
生成开始于 2:20...
1>----- 已启动生成: 项目: demo-cpp, 配置: Debug Win32 -----
1>demo.cpp
1>D:\Workspace\VS2022-demo\demo-cpp\demo.cpp(6,1): warning C4723: 潜在的被 0 除
1>demo-cpp.vcxproj -> D:\Workspace\VS2022-demo\Debug\demo-cpp.exe
1>已完成生成项目“demo-cpp.vcxproj”的操作。
生成: 1 成功, 0 失败, 0 最新, 0 已跳过
生成于 2:20 完成, 耗时 01.124 秒
```

A red box highlights the warning message: `warning C4723: 潜在的被 0 除`.

The screenshot shows the Visual Studio IDE with a C++ file named `demo.cpp`. The code is as follows:

```
1 #include <iostream>
2 using namespace std;
3 int main()
4 {
5     float f1 = 1.2F;
6     float f2 = 0;
7     cout << f1 / f2 << endl;
8
9     return 0;
10 }
```

The output window shows the following message:

```
生成开始于 2:22...
1>----- 已启动生成: 项目: demo-cpp, 配置: Debug Win32 -----
1>demo.cpp
1>D:\Workspace\VS2022-demo\demo-cpp\demo.cpp(6,1): warning C4723: 潜在的被 0 除
1>demo-cpp.vcxproj -> D:\Workspace\VS2022-demo\Debug\demo-cpp.exe
1>生成: 1 成功, 0 失败, 0 最新, 0 已跳过
生成于 2:22 完成, 耗时 01.171 秒
```

A red box highlights the warning message: `warning C4723: 潜在的被 0 除`.



§. 习题课 (Week-06/07)

5、其他

★ 作业中部分数据的类型问题

问题：按课件说法，不加U/L等后缀的整型，默认应该是int型，然而某些情况下并不是

例1:

作业已结束

W0301-变量及表达式求值

问题 #29 (分数3)

求表达式的值（要求给出计算过程、每步计算结果及数据类型、对应的vs编译器下的验证程序及结果截图）

`long(2.8F + 3.3) * 2 + (int)1.9 % 7U - 'p' * 2UL`

```
demo.cpp  (全局范围)  main()
1  #include <iostream>
2  using namespace std;
3  int main()
4  {
5      cout << typeid(long(2.8F + 3.3) * 2 + (int)1.9 % 7U - 'p' * 2UL).name() << endl;
6      cout << typeid(long(2.8F + 3.3) * 2 + (int)1.9 % 7U).name() << endl;
7      return 0;
8  }
9
```

按整型提升规则，第一个是UL
但为什么第2个也是UL而不是L?

Microsoft Visual Studio 调试控制台

```
unsigned long
unsigned long
```




§. 习题课 (Week-06/07)

5、其他

★ 作业中部分数据的类型问题

问题：按课件说法，不加U/L等后缀的整型，默认应该是int型，然而某些情况下并不是

例2：一个(int上限+1 ~ u_int上限)之间的数，不同进制、加负号

```
#include <iostream>
using namespace std;

int main()
{
    /* int上限是 0x7FFFFFFF, 上限+1且16进制 */
    cout << 0x80000000 << endl;
    cout << typeid(0x80000000).name() << endl;
    cout << int(0x80000000) << endl;

    return 0;
}
```

```
demo.cpp demo.cpp (全局范围)
1 #include <iostream>
2 using namespace std;
3
4 int main()
5 {
6     /* int上限是 0x7FFFFFFF, 上限+1且16进制 */
7     cout << 0x80000000 << endl;
8     cout << typeid(0x80000000).name() << endl;
9     cout << int(0x80000000) << endl;
10
11     return 0;
12 }
```

Microsoft Visual Studio 调试控制台

2147483648
unsigned int
-2147483648

```
#include <iostream>
using namespace std;

int main()
{
    /* int上限是 2147483647, 上限+1且10进制 */
    cout << 2147483648 << endl;
    cout << typeid(2147483648).name() << endl;
    cout << int(2147483648) << endl;

    return 0;
}
```

```
demo.cpp demo.cpp (全局范围)
1 #include <iostream>
2 using namespace std;
3
4 int main()
5 {
6     /* int上限是 2147483647, 上限+1且10进制 */
7     cout << 2147483648 << endl;
8     cout << typeid(2147483648).name() << endl;
9     cout << int(2147483648) << endl;
10
11     return 0;
12 }
```

Microsoft Visual Studio 调试控制台

2147483648
int64
-2147483648



§. 习题课 (Week-06/07)

5、其他

★ 作业中部分数据的类型问题

问题：按课件说法，不加U/L等后缀的整型，默认应该是int型，然而某些情况下并不是

例2：一个(int上限+1 ~ u_int上限)之间的数，不同进制、加负号

```
#include <iostream>
using namespace std;

int main()
{
    /* int下限且10进制 */
    cout << -2147483648 << endl;
    cout << typeid(-2147483648).name() << endl;
    cout << int(-2147483648) << endl;

    return 0;
}
```

```
#include <iostream>
using namespace std;

int main()
{
    /* 对int上限+1且16进制表示的加负号 */
    cout << -0x80000000 << endl;

    return 0;
}
```

The screenshot shows the Visual Studio IDE with a C++ file named 'demo.cpp'. The code is the same as the first example. The output window on the right shows the results of the program execution:

```
-2147483648
int64
-2147483648
```

The screenshot shows the Visual Studio IDE with a C++ file named 'demo.cpp'. The code is the same as the second example. The output window on the right shows the results of the program execution:

```
生成开始于 3:02...
1> 已启动生成: 项目: demo-cpp, 配置: Debug Win32
1> demo-cpp
1> D:\Workspace\VS2022-demo\demo-cpp\demo-cpp (7, 13): error C4146: 一元负运算符应用于无符号类型, 结果仍为无符号类型
1> 已完成生成项目 "demo-cpp.vcxproj" 的操作 - 失败。
生成: 0 成功, 1 失败, 0 最新, 0 已跳过
生成于 3:02 完成, 耗时 00.910 秒
```



§. 习题课 (Week-06/07)

5、其他

★ 作业中部分数据的类型问题

问题：按课件说法，不加U/L等后缀的整型，默认应该是int型，然而某些情况下并不是

答案：整型不带后缀的情况下，有一个从小到大的**梯次适应**规则，超过小数据的范围则自动递增一级

```
#include <iostream>
using namespace std;

int main()
{
    cout << typeid(0x80000000).name() << endl;
    cout << typeid(0x100000000).name() << endl;

    return 0;
}
```

二进制，八进制或十六进制底

- int
- unsigned int ←
- long int
- unsigned long int
- long long int (C++11 起) ←
- unsigned long long int (C++11 起)

demo.cpp

```
1 #include <iostream>
2 using namespace std;
3
4 int main()
5 {
6     cout << typeid(0x80000000).name() << endl;
7     cout << typeid(0x100000000).name() << endl;
8     return 0;
9 }
10
```

Microsoft Visual Studio 调试控制台

```
unsigned int
__int64
```

字面量的类型

整数字面量的类型，是依赖于所用数字基底和 **整数后缀** 确定的列表中，首个能适合其值的类型。

后缀	十进制底	二进制，八进制或十六进制底
(无后缀)	• int • long int • long long int (C++11 起)	• int • unsigned int • long int • unsigned long int • long long int (C++11 起) • unsigned long long int (C++11 起)
u 或 U	• unsigned int • unsigned long int • unsigned long long int (C++11 起)	• unsigned int • unsigned long int • unsigned long long int (C++11 起)
l 或 L	• long int • unsigned long int (C++11 前) • long long int (C++11 起)	• long int • unsigned long int • long long int (C++11 起) • unsigned long long int (C++11 起)
同时有 l/L 和 u/U	• unsigned long int • unsigned long long int (C++11 起)	• unsigned long int • unsigned long long int (C++11 起)
ll 或 LL	• long long int (C++11 起)	• long long int (C++11 起) • unsigned long long int (C++11 起)
同时有 ll/LL 和 u/U	• unsigned long long int (C++11 起)	• unsigned long long int (C++11 起)
z 或 Z	• std::size_t 的有符号版本 (C++23 起)	• std::size_t 的有符号版本 (C++23 起) • std::size_t (C++23 起)
同时有 z/Z 和 u/U	• std::size_t (C++23 起)	• std::size_t (C++23 起)

若不具有 **大小后缀** 的 (C++23 起) 整数字面量的值过大而无法符合任何后缀/底组合所允许的类型，且编译器支持能表示该字面量的值的扩展整数类型（如 `__int128`），则字面量可以被授予该扩展整数类型——否则程序非良构。



§ . 习题课 (Week-06/07)

5、其他

★ 作业中部分数据的类型问题

问题：按课件说法，不加U/L等后缀的整型，默认应该是int型，然而某些情况下并不是

答案：整型不带后缀的情况下，有一个从小到大的**梯次适应**规则，超过小数据的范围则自动递增一级

再次强调：限于课时，课上所讲的只是一个简单规则，更复杂的表现并未进一步讨论，一旦出现不同，**编译器具体表现为准**

● **VS应用小技巧：鼠标移动到数据/运算符上，会显示类型**



§. 习题课 (Week-06/07)

5、其他

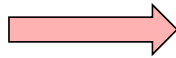
★ 未定义行为 (UB行为 = Undefined Behavior)

引入：为什么不同编译器处理方式不同？

回答：C/C++的标准是一套语法定义，VS/gcc是对这个标准的具体实现，标准要求**正确处理的情况下结果相同**，但对于**具体的实现过程**可能有强制规定，也可能允许自行定义 (ID=**I**mplement **D**efined)，实现过程如果遇到错误也允许实现者自行决定，称为未定义行为 (UB = **U**ndefined **B**ehavior)

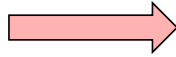
例1：让VS和gcc去KFC买一个香辣鸡腿堡（正确行为），有货

强制要求必须线下使用9折校园卡



最终结果：两者都是正确结果
购买过程：实现过程强制规定

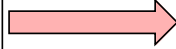
VS：手机APP，线上点单
gcc：9折校园卡，线下排队



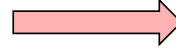
最终结果：两者都是正确结果
购买过程：**I**mplement **D**efined

例2：让VS和gcc去KFC买一个香辣鸡腿堡（正确行为），没货

缺货(错误处理)



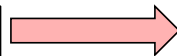
VS：改买劲脆鸡腿堡
gcc：不吃了，饿一顿



最终结果：UB
但都合理

例3：让VS和gcc去KFC买一个**麦当劳的麦辣**鸡翅（错误行为）

错误行为



VS：今晚睡KFC了
gcc：跑到路上把垃圾箱踢翻



最终结果：UB
重点：完全超出使用者预期的结果